

tf.compat.v1.nn.safe_embedding_lookup_sparse

Stay organized with collections Save and categorize content based on your preferences.

https://www.tensorflow.org/api_docs/python/tf/compat/v1/nn/safe_embedding_lookup_sparse

Lookup embedding results, accounting for invalid IDs and empty features.

Function Signatures

```
tf.compat.v1.nn.safe_embedding_lookup_sparse(  
    embedding_weights, sparse_ids, sparse_weights=None,  
    combiner='mean', default_id=None, name=None,  
    partition_strategy='div', max_norm=None,  
    allow_fast_lookup=False )
```

Code Examples

```
tf.compat.v1.nn.safe_embedding_lookup_sparse(  
embedding_weights,  
sparse_ids,  
sparse_weights=None,  
combiner='mean',  
default_id=None,  
name=None,  
partition_strategy='div',  
max_norm=None,  
allow_fast_lookup=False  
)
```

```
tf.compat.v1.nn.safe_embedding_lookup_sparse(  
embedding_weights,  
sparse_ids,  
sparse_weights=None,  
combiner='mean',  
default_id=None,  
name=None,  
partition_strategy='div',  
max_norm=None,  
allow_fast_lookup=False  
)
```

```
[0, 0]: id 1, weight 2.0  
[0, 1]: id 3, weight 0.5  
[1, 0]: id -1, weight 1.0  
[2, 3]: id 1, weight 3.0
```

```
[0, 0]: id 1, weight 2.0  
[0, 1]: id 3, weight 0.5  
[1, 0]: id -1, weight 1.0  
[2, 3]: id 1, weight 3.0
```

```
output[0, :] = (params[1, :] * 2.0 + params[3, :] * 0.5) / (2.0 + 0.5)
output[1, :] = (params[0, :] * 1.0) / 1.0
output[2, :] = (params[1, :] * 3.0) / 3.0
```

```
output[0, :] = (params[1, :] * 2.0 + params[3, :] * 0.5) / (2.0 + 0.5)
output[1, :] = (params[0, :] * 1.0) / 1.0
output[2, :] = (params[1, :] * 3.0) / 3.0
```

Documentation

Lookup embedding results, accounting for invalid IDs and empty features.

The partitioned embedding in `embedding_weights` must all be the same shape except for the first dimension. The first dimension is allowed to vary as the vocabulary size is not necessarily a multiple of `P`. `embedding_weights` may be a `PartitionedVariable` as returned by using `tf.compat.v1.get_variable()` with a `partitioner`.

Invalid IDs (< 0) are pruned from input IDs and weights, as well as any IDs with non-positive weight. For an entry with no features, the embedding vector for `default_id` is returned, or the 0-vector if `default_id` is not supplied.

The ids and weights may be multi-dimensional `SparseTensors` or `RaggedTensors` with rank of 2. For `SparseTensors` with left-aligned non-zero entries which can be described as `RaggedTensors`, use of `RaggedTensors` can yield higher performance. Embeddings are always aggregated along the last dimension.

Returns

In other words, if

`shape(combined embedding_weights) = [p0, p1, ..., pm]`

`shape(sparse_ids) = shape(sparse_weights) = [d0, d1, ..., dn]`

`shape(output) = [d0, d1, ... dn-1, p1, ..., pm]`.

For instance, if `params` is a 10x20 matrix, and `sp_ids` / `sp_weights` are

`default_id` is 0.

with `combiner="mean"`, then the output will be a 3x20 matrix where

`## Raises`